MAJOR TECHNICAL PROJECT | E2EE CRYPTOGRAPHY | ADAPTIVE MACHINE LEARNING

# NearShare: System Architecture & Engineering Documentation

A High-Throughput Peer-to-Peer File Transfer System Engineered with Zero-Knowledge ECDH Key Agreement, AES-256-GCM Authenticated Encryption, and k-NN Telemetry Optimization.

PROJECT REPOSITORY
github.com/PranjalSingh13/nearshare

CORE TECH STACK
React 19, Node.js, Express, WebSockets, SQLite3

CRYPTOGRAPHIC STANDARD
ECDH (prime256v1) + AES-256-GCM + SHA-256

TARGET CAPACITY
Up to 50 GB Payloads (Local LAN & Tunneling)

## PROJECT TEAM MEMBERS & ENGINEERING RESPONSIBILITIES

MEMBER 1 • FULL-STACK ARCHITECT & LEAD

### Pranjal Singh

System design, Node.js Express server, full-duplex WebSocket streaming protocol, Multer 50GB file staging pipeline, and concurrent workspace orchestration.

MEMBER 2 • SECURITY & CRYPTOGRAPHY SPECIALIST

### Team Member 2

Zero-knowledge ECDH (prime256v1) key agreement, AES-256-GCM chunk cipher/decipher implementation, 12-byte IV rotation, and SHA-256 integrity digests.

MEMBER 3 • MACHINE LEARNING & OPTIMIZATION

### Team Member 3

Embedded k-Nearest Neighbors (k-NN) engine, SQLite telemetry logging schema, dynamic chunk size optimization (256KB-2MB), and heuristic compression bypass.

MEMBER 4 • FRONTEND UI/UX & WEB CRYPTO

### Team Member 4

React 19 single-page app, QR code generation and mobile peer pairing, ClientView in-browser Web Crypto API stream decryption, and real-time telemetry dashboard.

# 1. Executive Summary & Problem Formulation

In the contemporary digital landscape, sharing large files (such as high-definition 4K video footage, virtual machine images, game assets, and zipped data archives) between nearby devices remains surprisingly inefficient and insecure. Users are typically forced into one of two suboptimal workflows:

1. **Cloud Relay Services (Google Drive, WeTransfer, Dropbox)**: Require uploading files to third-party data centers before the adjacent peer can download them. This wastes substantial broadband bandwidth, incurs cloud storage limits, violates data privacy, and creates single points of failure.
2. **Consumer Ad-Hoc Solutions (AirDrop, Quick Share)**: Are strictly walled-garden ecosystems, failing when attempting to transfer cross-platform between Windows, Linux, iOS, and Android without proprietary vendor drivers.

**NearShare** resolves these fundamental bottlenecks by providing an operating-system-agnostic, zero-cloud peer-to-peer file transfer engine. Operating over the local area network (LAN) with fallback hybrid cloud tunneling (via ngrok), NearShare combines browser-native cryptography, real-time WebSocket pipelining, and continuous machine-learning-driven chunk adaptation to achieve maximum line-rate network utilization.

> **Core Engineering Objectives**
>
> • **Zero-Knowledge Privacy**: The server acts strictly as an untrusted signaling and chunk router; only the host and verified client hold derived encryption keys.
> • **Line-Rate Local Throughput**: Bypasses external internet bottlenecks, transferring at full local router speeds (up to gigabit LAN limits).
> • **Frictionless UX**: No native software installations; pairing occurs in under 3 seconds via dynamic QR codes scanned on any smartphone camera.
> • **High-Capacity Reliability**: Supports payloads up to 50 GB with resumable chunk status tracking backed by an embedded SQLite database.

---

# 2. End-to-End System Architecture

NearShare is decoupled into three primary architectural tiers: the **Host Node**, the **Signaling & Storage Engine (Backend)**, and the **Receiver Node (Client)**.

### 2.1 Component Interaction Topology

| LAYER | TECHNOLOGIES | KEY RESPONSIBILITIES |
|---|---|---|
| **Frontend Client & Host Dashboard** | React 19, Vite, Lucide-React, QRCode | Local IP autodiscovery, QR presentation, drag-and-drop staging, real-time progress bars, and in-browser Web Crypto API stream decryption. |
| **Signaling & API Gateway** | Express.js, WebSockets (`ws`) | Session lifecycle management, peer discovery, WebSocket event broadcasting, and chunk stream distribution. |

| LAYER | TECHNOLOGIES | KEY RESPONSIBILITIES |
| --- | --- | --- |
| **Cryptographic Layer** | Node.js `crypto` & Web Crypto API | ECDH prime256v1 key exchange, SHA-256 key derivation, AES-256-GCM chunk encryption, and SHA-256 data integrity checks. |
| **Adaptive ML Engine** | Custom k-NN in Node.js, `zlib` | Queries telemetry logs to predict optimal chunk payload size (256 KB – 2 MB+) and selectively bypasses compression for media files. |
| **Persistence & State Engine** | SQLite3 ( `nearshare.db` ) | Tracks session tokens, chunk-level delivery status, and historical performance logs for ML training. |

# 3. Cryptographic Architecture & Security Model

NearShare implements a defense-in-depth, zero-knowledge cryptographic model. Files are never stored or transmitted in plaintext. The cryptographic lifecycle consists of four distinct phases:

### Phase 1: Ephemeral Key Exchange via ECDH

The host and client each generate an independent Elliptic Curve Diffie-Hellman (ECDH) key pair using the NIST standard curve `prime256v1` (P-256):

```
// Host generates ECDH Keypair
const ecdh = crypto.createECDH('prime256v1');
ecdh.generateKeys();
const hostPublicKeyHex = ecdh.getPublicKey('hex');
```

When the client connects (via QR code or join link), the client transmits its public key. Both parties independently compute the identical shared secret $S$:

$$S = ext{ECDH}_{ ext{private}}( ext{Peer}_{ ext{public}})$$

### Phase 2: Symmetric Key Derivation (KDF)

To prevent weak-key vulnerabilities, the raw elliptic curve shared secret is compressed and diffused into a uniform 256-bit (32-byte) symmetric key using SHA-256:

```
// Derive uniform 256-bit AES Key
const aesKey = crypto.createHash('sha256').update(sharedSecret).digest();
```

### Phase 3: Chunk-Level Authenticated Encryption (AES-256-GCM)

Unlike basic CBC or CTR modes, **Galois/Counter Mode (GCM)** provides Authenticated Encryption with Associated Data (AEAD). Each chunk receives:

- A unique, cryptographically secure 12-byte Initialization Vector (IV).
- A 16-byte Authentication Tag generated by the GCM cipher.

```
export function encryptChunk(key, payload) {
  const iv = crypto.randomBytes(12); // Unique 12-byte IV per chunk
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
  const encrypted = Buffer.concat([cipher.update(payload), cipher.final()]);
  const tag = cipher.getAuthTag(); // 16-byte authentication tag
  return { iv: iv.toString('hex'), tag: tag.toString('hex'), ciphertext: encrypted.toString('hex') };
}
```

### Phase 4: Bit-Level Integrity Verification

Before saving or assembling the stream on the receiver, each chunk's SHA-256 digest is validated against the status table. If a single bit is flipped due to network jitter or deliberate tampering, the GCM auth tag verification fails immediately, triggering an automatic chunk re-request.

| THREAT VECTOR | MITIGATION MECHANISM IN NEARSHARE |
| --- | --- |
| **Passive Eavesdropping (Packet Sniffing)** | All payloads are encrypted with AES-256-GCM. An attacker capturing raw Wi-Fi packets sees only ciphertext. |
| **Man-in-the-Middle (MitM) Alteration** | GCM authentication tags fail decryption if ciphertext or headers are tampered with. |
| **Replay Attacks** | Every chunk generates an independent 12-byte random IV; old chunks cannot be injected into future sessions. |
| **Compromise of Historical Sessions** | Ephemeral ECDH keys are never written to disk; sessions enjoy Forward Secrecy. |

# 4. Adaptive Machine Learning Optimizer

A primary challenge in high-speed file transfer is selecting the optimal chunk size. If chunks are too small (e.g., 32 KB), network socket overhead and WebSocket framing headers cause high CPU utilization and degraded throughput. If chunks are too large (e.g., 8 MB), socket buffering stalls, and packet loss necessitates expensive retransmissions.

### 4.1 The k-Nearest Neighbors (k-NN) Engine

NearShare implements an internal k-NN machine learning predictor in `ml_optimizer.js`. Rather than relying on rigid static rules, the optimizer queries the SQLite `telemetry` table to evaluate previous transfer conditions.

> **Mathematical Distance Formulation**
>
> The model normalizes file size into a logarithmic scale ($\log_{10}$) to ensure compatibility with client count dimensions. For a new transfer with file size $S$ and client count $C$, the Euclidean distance to historical record $i$ is calculated as:

$$D_i = \sqrt{ \left( \log_{10}(S) - \log_{10}(S_i) ight)^2 + (C - C_i)^2 }$$

### 4.2 Optimization Algorithm Workflow

1. **Feature Extraction**: Measures target file size in bytes and number of concurrently connected peers.
2. **Euclidean Distance Ranking**: Computes $D_i$ for all historical telemetry records and sorts by proximity.
3. **Top-K Neighborhood Selection**: Selects the nearest $K = 5$ neighbor transfer logs.
4. **Throughput Maximization**: Within the $K$ neighborhood, identifies the record that achieved the highest throughput ($ ext{throughput_mbps}$).
5. **Parameter Assignment**: Applies the winning `chunk_size_used` and `parallel_streams_used` to the new session.

```
// k-NN Prediction Logic (ml_optimizer.js)
const scoredLogs = logs.map(log => {
  const logSizeLog = Math.log10(log.file_size || 1);
  const logClients = log.num_clients || 1;
  const distance = Math.sqrt(Math.pow(targetSizeLog - logSizeLog, 2) + Math.pow(targetClients - logClie
  return { log, distance };
});

scoredLogs.sort((a, b) => a.distance - b.distance);
const nearestNeighbors = scoredLogs.slice(0, Math.min(5, scoredLogs.length)).map(x => x.log);

// Select parameters that achieved highest historical throughput
let bestConfig = nearestNeighbors.reduce((prev, curr) =>
  (curr.throughput_mbps > prev.throughput_mbps) ? curr : prev
);
```

### 4.3 Content-Aware Compression Heuristics

Attempting to compress formats that are already internally compressed (such as `.mp4`, `.mkv`, `.png`, `.zip`, `.tar.gz`, `.pdf`) burns excessive CPU cycles without reducing payload size. NearShare cross-references file extensions against an internal registry (`ALREADY_COMPRESSED_EXT`), immediately disabling gzip compression for pre-compressed assets.

# 5. Backend Architecture & Database Schema

The backend is powered by Node.js using ES Modules, Express, Multer, and the `ws` library. It integrates an SQLite3 relational storage layer for ACID-compliant transfer state tracking.

### 5.1 Database Schema (`nearshare.db`)

| TABLE NAME | KEY COLUMNS | DESCRIPTION |
|---|---|---|
| `sessions` | `session_id` (PK), `file_path`, `file_name`, `file_size`, `chunk_size`, `total_chunks`, `created_at` | Maintains active and staged transfer sessions, chunk counts, and file metadata. |
| `chunk_status` | `client_id`, `session_id`, `chunk_index`, `status` ('transferred', 'pending'), `sha256` | Provides granular chunk-level delivery tracking. Enables interrupted transfers to resume instantly without restarting. |
| `telemetry` | `id` (PK), `file_extension`, `file_size`, `num_clients`, `chunk_size_used`, `compression_used`, `throughput_mbps`, `timestamp` | Continuous training dataset queried by the k-NN model to forecast optimal parameters for subsequent runs. |

### 5.2 High-Capacity File Ingestion (Up to 50 GB)

NearShare configures Multer disk storage to stream multi-gigabyte payloads straight to disk in `shared_files/` without exceeding Node's memory heap:

```
const upload = multer({
  storage: diskStorage,
  limits: { fileSize: 50 * 1024 * 1024 * 1024 } // 50 GB threshold
});
```

---

# 6. Frontend & Mobile Integration

Built with **React 19** and **Vite 5.4**, the user interface emphasizes rapid peer connection and responsive visual feedback.

### 6.1 Dual Mode Operating Flow

- **Host Send Mode (`HostDashboard.jsx`)**: The user selects or drags files into the staging zone. The dashboard calculates cryptographic digests, provisions an ECDH session token, displays real-time line charts of network speed, and generates a dynamic QR code containing the pairing URL: `http://[Local-IP]:[Port]?join=[SessionID]`.
- **Host Receive Mode**: Operates as an open local dropbox. Mobile clients scan the QR code to push photos, videos, and files directly to the host's hard drive.

- **Client Join Mode (`ClientView.jsx`)**: Scans or visits the join URL. Leverages the browser's native **Web Crypto API** (`window.crypto.subtle`) to compute the shared secret, decrypt incoming binary chunks on the fly, and reconstruct the final file as an in-memory `Blob` for instantaneous one-tap download.

# 7. Team Member Contribution Matrix

This project was designed, implemented, and benchmarked by a 4-member engineering team. The specific breakdown of deliverables and individual ownership is detailed below:

| TEAM MEMBER | PRIMARY ROLE | SPECIFIC MODULES & DELIVERABLES |
|---|---|---|
| **Member 1 (Lead)** Pranjal Singh | Full-Stack Architect & Core Infrastructure | • Designed end-to-end WebSocket protocol ( `server.js` ).<br>• Implemented 50 GB Multer disk streaming pipeline.<br>• Configured multi-package workspace orchestration ( `concurrently` ).<br>• Implemented ngrok dynamic tunneling integration for remote WAN sharing. |
| **Member 2** Team Member 2 | Cryptography & Security Engineer | • Authored `crypto.js` : ECDH prime256v1 key exchange.<br>• Built AES-256-GCM chunk cipher with dynamic 12-byte IV rotation.<br>• Implemented SHA-256 key derivation & chunk validation digests.<br>• Conducted cryptographic threat modeling & security audits. |
| **Member 3** Team Member 3 | Machine Learning & Telemetry Specialist | • Formulated and coded the k-NN optimization engine ( `ml_optimizer.js` ).<br>• Designed SQLite3 relational schema for telemetry and chunk state ( `db.js` ).<br>• Developed content-aware compression bypass algorithms.<br>• Created the automated integration test suite ( `test_transfer.js` ). |
| **Member 4** Team Member 4 | Frontend UI/UX & Web Crypto Specialist | • Engineered React 19 single-page UI ( `App.jsx` , `HostDashboard.jsx` ).<br>• Implemented browser Web Crypto API stream decryption in `ClientView.jsx` .<br>• Built QR code pairing, copyable network links, and mobile responsiveness.<br>• Designed dark-mode visual hierarchy and animated SVG indicators. |

---

# 8. Verification, Testing & Future Roadmap

### 8.1 Automated Test Verification

NearShare includes an automated end-to-end transfer verification script ( `test_transfer.js` ). The test harness creates dummy binary payloads, triggers cryptographic key generation, performs chunked AES-256-GCM encryption, verifies SHA-256 hash matching before and after reassembly, and logs benchmark throughput to the telemetry database.

### 8.2 Future Roadmap

1. **WebRTC DataChannels**: Upgrading direct peer connections to WebRTC DataChannels for direct NAT-traversed browser-to-browser streaming without routing through local Node.js sockets.
2. **Mesh Swarm Distribution**: Implementing BitTorrent-style piece sharing, allowing multiple receiving clients to share pieces between each other to eliminate host bandwidth bottlenecks.

> **Conclusion**
>
> NearShare demonstrates that modern web standards (React 19, WebSockets, Web Crypto API) coupled with adaptive machine learning can outperform traditional cloud relays and proprietary ad-hoc protocols—delivering exceptional speed, zero-cloud privacy, and platform independence.